Forensics CTF

Challenge Writeup Report

Student: Elvis Avduli

AM: E22001

Course: Digital Forensics / CTF Lab

Date: June 9, 2026

Contents

1 Introduction **2**

2 Challenge 1: Babushka **2**
2.1 Challenge Description . . . 2
2.2 Initial Observations . . . 2
2.3 Solution Process . . . 2
2.3.1 Step 1 — Verify File Type . . . 2
2.3.2 Step 2 — Scan with Binwalk . . . 2
2.3.3 Step 3 — Extract Embedded Files . . . 3
2.3.4 Step 4 — Navigate the Nested Layers . . . 3
2.3.5 Step 5 — Extract the Final Archive and Read the Flag . . . 3
2.4 Results . . . 3

3 Challenge 2: Hacked **4**
3.1 Challenge Description . . . 4
3.2 Initial Observations . . . 4
3.3 Solution Process . . . 4
3.3.1 Step 1 - Inspect the End of the File with xxd . . . 4
3.4 Results . . . 4

4 Challenge 3: Information **5**
4.1 Challenge Description . . . 5
4.2 Initial Observations . . . 5
4.3 Solution Process . . . 5
4.3.1 Step 1 - Dump All Metadata with exiftool . . . 5
4.3.2 Step 2 - Decode the ROT13 Encoding . . . 5
4.4 Results . . . 6

5 Challenge 4: Headstream **7**
5.1 Challenge Description . . . 7
5.2 Initial Observations . . . 7
5.3 Solution Process . . . 7
5.3.1 Step 1 - Extract the Encrypted ExeScript Payload . . . 7
5.3.2 Step 2 - Decrypt the Payload . . . 7
5.3.3 Step 3 - Clean and View the Decrypted Script . . . 7
5.4 Issue Found . . . 8
5.5 Evidence . . . 9
5.6 Conclusion . . . 9

1. Introduction

This report documents the complete analysis and solution methodology applied to three image-based steganography and digital forensics challenges from the CTFLib platform. Each challenge requires the discovery of a hidden flag in the form `CTFLib{...}` concealed within image files using different data-hiding techniques.

The report is structured as a technical writeup: for each challenge, initial observations, analytical steps, tools used, commands executed, and the resulting flag are described in full detail.

2. Challenge 1: Babushka

2.1 Challenge Description

The challenge presents an image of a Russian matryoshka (nesting) doll alongside books. The description states: *"just like real Babushka dolls, this image contains layers to find what it really hides."* This is a direct metaphor for **nested file embedding** — a file hidden inside another file, potentially multiple levels deep.

2.2 Initial Observations

- The image is a standard-looking JPEG photograph.
- The metaphor of "layers" strongly suggests embedded files via `binwalk`-detectable signatures.
- The file size may be anomalously large for its visual content, a common indicator of appended or embedded data.

2.3 Solution Process

2.3.1 Step 1 — Verify File Type

```
file Babushka.jpg
```

Output confirmed a legitimate JPEG file (JFIF standard 1.01, 3946x5919, progressive, 72 DPI). No extension spoofing was detected, but the unusually large file size for a photograph suggested appended or embedded data.

2.3.2 Step 2 — Scan with Binwalk

```
binwalk Babushka.jpg
```

`binwalk` revealed the following embedded content starting at offset `0x382204` (3,678,724 bytes into the file):

- `Layer_1/Layer_2/` — nested ZIP directory
- `Layer_1/Layer_2/Layer_3/` — deeper nested directory

- `Layer_1/Layer_2/Layer_3/Flag.zip` — innermost archive (208 bytes compressed)
- Two ZIP End-of-Archive footers confirming the structure

This perfectly matched the matryoshka metaphor: three layers deep, each containing the next.

2.3.3 Step 3 — Extract Embedded Files

```
binwalk -e Babushka.jpg
```

`binwalk` extracted the contents into `_Babushka.jpg.extracted/`, producing the directory `382204.zip` and a `Layer_1` folder. A warning was raised that one file failed to extract automatically (`Flag.zip`), requiring manual extraction in the next step.

2.3.4 Step 4 — Navigate the Nested Layers

```
cd _Babushka.jpg.extracted/
ls                          # reveals: 382204.zip  Layer_1/
cd Layer_1/Layer_2/Layer_3/
ls                          # reveals: Flag.zip
```

Navigating through the directory tree confirmed the three-layer nesting structure: `Layer_1` → `Layer_2` → `Layer_3` → `Flag.zip`.

2.3.5 Step 5 — Extract the Final Archive and Read the Flag

```
unzip Flag.zip
# Output: extracting: Flag.txt
cat Flag.txt
```

2.4 Results

Unzipping `Flag.zip` produced `Flag.txt`, which contained the flag in plaintext.

Flag Found

`CTFLib{You_have_unfolded_the_secrets_of_the_Russian_doll!}`

3. Challenge 2: Hacked

3.1 Challenge Description

The challenge presents an image displaying source code and a padlock icon, appearing visually normal. The description states: *"can you find if it has indeed been modified and find the hidden secret?"* This implies the image has been **steganographically altered** — either through LSB encoding, pixel manipulation, or data appended after the end-of-file marker.

3.2 Initial Observations

- The image appears to be a PNG or JPEG with a security/hacking theme.
- The padlock graphic is likely a red herring — the hidden data is at the binary level.
- Visual normalcy despite claimed modification points toward LSB steganography or appended data.

3.3 Solution Process

3.3.1 Step 1 - Inspect the End of the File with xxd

Every JPEG file ends with the two-byte **EOI (End of Image)** marker `FF D9`. Any bytes that exist beyond this marker are completely invisible to image viewers, but are physically present in the file. This makes post-EOF data a simple yet effective method to hide information. To inspect the final bytes of the file, the `xxd` hex dump tool was used, piped through `tail` to show only the last 50 lines:

```
xxd Hacked.jpg | tail -50
```

Scanning the output carefully, the following bytes were found at offset `0x3f183`:

```
0003f180: 873f ff00 4354 464c 6962 7b54 6869 735f  .?..CTFLib{This_
0003f190: 6973 5f74 6865 5f66 6c61 675f 6869 6464  is_the_flag_hidd
0003f1a0: 656e 5f69 6e5f 6865 785f 666f 726d 217d  en_in_hex_form!}
```

The flag was appended as plaintext after the JPEG image data ended, encoded directly in the raw bytes of the file. The ASCII representation on the right column of the hex dump makes it immediately readable.

3.4 Results

The flag was discovered by inspecting the raw hex dump of the file, appended after the JPEG EOI marker with no additional encoding or compression.

Flag Found

`CTFLib{This_is_the_flag_hidden_in_hex_form!}`

4. Challenge 3: Information

4.1 Challenge Description

The challenge presents a photograph of a German Shorthaired Pointer dog. The description states the dog is *"lost"* and that we need to *"find the owner to return it."* This is a deliberate hint toward **EXIF metadata**: just as a lost dog's tag holds owner information, the image file's metadata fields hold the hidden flag.

4.2 Initial Observations

- The image is a high-quality JPEG photograph — likely rich in EXIF metadata.
- Typical JPEG metadata includes camera model, GPS coordinates, author, and freeform comment fields.
- The "owner" hint suggests fields like `Artist`, `Author`, `Copyright`, or `Comment`.
- This is the most straightforward of the three challenges but requires careful reading of all metadata fields.

4.3 Solution Process

4.3.1 Step 1 - Dump All Metadata with exiftool

```
exiftool Information.jpg
```

`exiftool` outputs every metadata field embedded in the file across all standards (EXIF, IPTC, XMP, ICC Profile). Scanning through the output, the flag string appeared in three separate metadata fields:

```
Artist                  : CTFLib{Gur_vasbezngvba_vf_va_gur_Zrgnqngn!}
By-line                 : CTFLib{Gur_vasbezngvba_vf_va_gur  (truncated in
    IPTC)
Creator                 : CTFLib{Gur_vasbezngvba_vf_va_gur_Zrgnqngn!}
Image Creator Name      : CTFLib{Gur_vasbezngvba_vf_va_gur_Zrgnqngn
   !}
```

The flag value `CTFLib{Gur_vasbezngvba_vf_va_gur_Zrgnqngn!}` was found in the **Artist** (EXIF), **Creator** (XMP), and **Image Creator Name** (XMP) fields. The **By-line** field (IPTC) contained a truncated version of the same string due to IPTC field length limitations.

4.3.2 Step 2 - Decode the ROT13 Encoding

The flag content inside the curly braces does not look like plain English — it is encoded with **ROT13**, a simple Caesar cipher that rotates each letter by 13 positions. ROT13 is self-inverse: applying it twice returns the original text. The encoded string was decoded as follows:

```
echo "Gur_vasbezngvba_vf_va_gur_Zrgnqngn!" | tr 'A-Za-z' 'N-ZA-
   Mn-za-m'
```

Output:

```
The_information_is_in_the_Metadata!
```

This confirms the flag and is consistent with the challenge hint: the "owner information" (i.e., the `Artist`/`Creator` metadata fields) reveals the secret once decoded.

4.4 Results

The flag was found in the `Artist`, `Creator`, and `Image Creator Name` EXIF/XMP metadata fields via `exiftool`, encoded in ROT13. After decoding, the full flag is:

Flag Found

`CTFLib{The_information_is_in_the_Metadata!}`

5. Challenge 4: Headstream

5.1 Challenge Description

The challenge provides an executable file named `GTAVI.exe`. The scenario suggests that a fake leaked version of GTA VI was downloaded and, when opened, only displayed an error message. Since this is a forensics challenge, the objective was to inspect the executable and identify where the hidden payload or flag was stored.

5.2 Initial Observations

- The file is a Windows executable.
- Running it is not necessary and could be unsafe, so I analyzed it statically.
- The executable appears to be an **ExeScript** wrapper, meaning it contains an embedded script.
- The embedded script was not visible through a normal `strings` search because it was encrypted.

5.3 Solution Process

5.3.1 Step 1 - Extract the Encrypted ExeScript Payload

The encrypted payload was extracted from the executable at the relevant offset:

```
dd if=GTAVI.exe of=encrypted_payload.bin bs=1 skip=$((0x19bc4))
    count=512 status=none
```

5.3.2 Step 2 - Decrypt the Payload

The payload was decrypted using AES-256-ECB with the known default ExeScript key:

```
openssl enc -d -aes-256-ecb \
-in encrypted_payload.bin \
-out decrypted_raw.bin \
-K
   4578655363726970740000000000000000000000000000000000000000000000
    \
-nopad
```

5.3.3 Step 3 - Clean and View the Decrypted Script

The real script length was extracted and Windows carriage returns were removed for readability:

```
head -c 506 decrypted_raw.bin > decrypted_payload.vbs
tr -d '\r' < decrypted_payload.vbs > decrypted_payload_clean.
   vbs
cat decrypted_payload_clean.vbs
```

The decrypted script contained the following important lines:

```
path = WScript.ScriptFullName
path = Replace(path,"vbs","exe")
CreateObject("WScript.Shell").Run "wscript "&path&":wininit.vbs
    "
```

5.4 Issue Found

The line:

```
wscript GTAVI.exe:wininit.vbs
```

indicates that the executable expects a second-stage script stored in an **NTFS Alternate Data Stream** named `wininit.vbs`. In Windows, the syntax `filename:streamname` refers to an alternate data stream attached to a file.

However, after extracting the provided archive, only the normal file `GTAVI.exe` was present. The expected stream `wininit.vbs` was not available as a normal file, and therefore the next stage of the challenge could not be recovered.

This was verified with:

```
ls -la
```

The directory contained only `GTAVI.exe` and the files I created during analysis, such as `encrypted_payload.bin` and `decrypted_payload_clean.vbs`. No `wininit.vbs` file or second-stage payload was present.

5.5 Evidence

```
┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ dd if=GTAVI.exe of=encrypted_payload.bin bs=1 skip=$((0×19bc4)) count=512 status=n
one

┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ openssl enc -d -aes-256-ecb -in encrypted_payload.bin -out decrypted_raw.bin -K 45
7865536372697074000000000000000000000000000000000000000000000000 -nopad

┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ head -c 506 decrypted_raw.bin > decrypted_payload.vbs

┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ tr -d '\r' < decrypted_payload.vbs > decrypted_payload_clean.vbs

┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ cat decrypted_payload_clean.vbs

' ——— ExeScript Options Begin ———




' ——— ExeScript Options End ———
x=MsgBox("The application was unable to start correctly (0×c000007b). Click OK to clos
e the application", vkOkOnly+vbCritical, "Application Error")

path = WScript.ScriptFullName
path = Replace(path,"vbs","exe")
CreateObject("WScript.Shell").Run "wscript "&path&":wininit.vbs"
┌──(N0tPresl3y㉿HP-Elvis)-[~/Forensics Exercises 2026/Challenges/forensics-headstream]
└─$ ls -la
total 128
drwxr-xr-x 2 N0tPresl3y N0tPresl3y   4096 Jun  3 14:53 .
drwxr-xr-x 3 N0tPresl3y N0tPresl3y   4096 May 23 16:49 ..
-rw-r--r-- 1 N0tPresl3y N0tPresl3y    494 Jun  3 14:53 decrypted_payload_clean.vbs
-rw-r--r-- 1 N0tPresl3y N0tPresl3y    506 Jun  3 14:53 decrypted_payload.vbs
-rw-r--r-- 1 N0tPresl3y N0tPresl3y    512 Jun  3 14:53 decrypted_raw.bin
-rw-r--r-- 1 N0tPresl3y N0tPresl3y    512 Jun  3 14:53 encrypted_payload.bin
-rwxr-xr-x 1 N0tPresl3y N0tPresl3y 105984 May 22 23:25 GTAVI.exe
```

5.6 Conclusion

Based on the decrypted script, the challenge appears to be missing a required second-stage payload. The embedded script clearly attempts to execute `GTAVI.exe:wininit.vbs`, but the provided archive does not contain or preserve this NTFS Alternate Data Stream.

Therefore, I could not complete the challenge or recover the flag because the necessary payload appears to be absent from the distributed challenge files.